

Comparative Study of Deep Q-Network (DQN) and Double Deep Q-Network (Double DQN) on CartPole-v1 and LunarLander-v3

Md. Sharhan Khan

ID: 1000061595

Section: 01

Masters of Science in Computer Science
and Engineering
BRAC University

Abu Sayed

ID: 1000061640

Section: 01

Masters of Engineering in Computer
Science and Engineering
BRAC University

Moinul Hossain

ID: 1000062870

Section: 01

Masters of Science in Computer Science
and Engineering
BRAC University

Abdullah Al Redwan

ID: 1000062833

Section: 01

Masters of Engineering in Computer
Science and Engineering
BRAC University

Abstract

Reinforcement learning enables an agent to learn decision-making policies by interacting with an environment, and Deep Q-Network (DQN) has become a widely used approach for value-based control; however, standard DQN is known to suffer from overestimation of action values, which motivated the development of Double Deep Q-Network (Double DQN) as a modification that separates action selection from action evaluation. In this project, both DQN and Double DQN were implemented using PyTorch with experience replay, target networks, epsilon-greedy exploration, and temporal-difference learning, and were then trained and evaluated on two control environments with different levels of difficulty, namely CartPole-v1 and LunarLander-v3, using average reward, reward standard deviation, average episode length, and success rate as the evaluation metrics. The results show that both algorithms achieved identical, near-perfect performance on CartPole-v1, with an average reward of 500.00, a reward standard deviation of 0.00, and a 100% success rate for both DQN and Double DQN, while on LunarLander-v3, DQN obtained an average reward of 268.37 with a 96% success rate, whereas Double DQN obtained a higher average reward of 272.32, a higher success rate of 100%, and a lower reward standard deviation of 23.32 compared to 34.02 for DQN, indicating that Double DQN produced a more accurate and more stable policy on the more complex environment, while both methods performed identically on the simpler one, although additional runs and tuning are still needed before drawing a general conclusion about the two algorithms.

1 Introduction

Many real-world control problems require an agent to choose a sequence of actions whose consequences unfold over time, such as balancing a physical system or piloting a vehicle to a target while managing momentum and orientation. Reinforcement learning (RL) provides a general framework for solving such problems through trial-and-error interaction with an environment, where an agent learns a policy that maximizes cumulative reward. In this project, the problem is to compare Deep Q-Network (DQN) and Double Deep Q-Network (Double DQN) for learning discrete-action control policies and to determine whether Double DQN can provide a practical performance advantage by reducing the overestimation of action values. This problem is

important because inaccurate value estimation can affect the quality and stability of learned policies, while reliable RL-based control can be useful in applications such as robotics, autonomous vehicles, and resource management

A substantial body of prior work has studied this overestimation problem and proposed extensions to DQN. Watkins and Dayan formalized Q-learning and proved its convergence under a lookup-table representation, establishing the theoretical basis that DQN later built on [2], though their proof does not extend to neural-network function approximation, which is precisely the setting where instability emerges. Mnih et al. introduced DQN itself, combining Q-learning with a deep convolutional network, experience replay, and a target network to achieve human-level performance across 49 Atari games [3]; their method was highly effective at learning directly from raw pixels, but the paper itself did not address the overestimation bias inherent to the max operator in the Q-learning target. Van Hasselt first identified and quantified this overestimation bias in a tabular setting with Double Q-learning [4], showing the bias can arise from noise or approximation error alone, though this tabular result did not demonstrate whether the effect mattered at the scale of deep networks. Van Hasselt, Guez, and Silver extended this idea into Double DQN, decoupling action selection from action evaluation in the deep RL setting, and showed it reduced overestimation and improved scores on many Atari games [5]; however, their evaluation used a single fixed Atari benchmark and did not examine how the benefit transfers to different types of control tasks or reward structures. Wang et al. proposed Dueling Network Architectures, which separate the estimation of state value and action advantage, improving learning efficiency especially when many actions have similar values [11], though this addresses representational efficiency rather than the overestimation problem directly. Schaul et al. introduced Prioritized Experience Replay, which samples important transitions more frequently to accelerate learning [12], but at the cost of added implementation complexity and additional hyperparameters that require tuning. Hessel et al. combined Double DQN, dueling networks, prioritized replay, multi-step learning, distributional RL, and noisy networks into Rainbow, showing that these improvements are largely complementary and together achieve state-of-the-art results [13]; however, because Rainbow combines six techniques at once, its ablation results make it difficult to isolate how much benefit Double DQN specifically contributes on its own. Sutton and Barto’s textbook provides the broader theoretical foundations of MDPs, value functions, and temporal-difference learning that underlie all of the above methods [1]. Taken together, this literature establishes that Double DQN and related extensions can improve over standard DQN, but most published comparisons are conducted on complex, high-dimensional Atari benchmarks, leaving open the question of how these methods compare on simpler, more interpretable control environments where the effect of a single architectural change can be observed directly.

This gap motivates the present work. Rather than evaluating DQN and Double DQN on a single complex benchmark as in most of the literature above, this project implements both algorithms under the same general training pipeline and directly compares them on two environments of deliberately different difficulty, CartPole-v1 and LunarLander-v3, isolating the effect of the double-estimator target itself rather than a bundle of simultaneous improvements as in Rainbow. CartPole provides a simple, fast-to-solve control task where any behavioral difference between the two algorithms should be easy to observe, while LunarLander introduces continuous multi-dimensional state and a more demanding control task, closer in spirit to the Atari domain where Double DQN was originally shown to help. By reporting average reward, reward standard deviation, average episode length, success rate, and training-time convergence behavior across both environments, this project aims to give a clearer, more interpretable picture of when and how the reduction in overestimation bias actually translates into a better learned policy, rather than relying on aggregate Atari scores alone.

1.1 Our Contribution

The main contributions of this project are:

- Implemented Deep Q-Network (DQN) and Double Deep Q-Network (Double DQN) using PyTorch.
- Evaluated both algorithms on CartPole-v1 and LunarLander-v3 environments.
- Defined the Markov Decision Processes (MDPs), neural-network architectures, replay memory, target networks, and training procedures.

- Recorded and analyzed reward, reward standard deviation, episode length, success rate, epsilon, and training loss.
- Compared the performance of DQN and Double DQN and analyzed their performance differences across the two environments.

2 Material and Methods

The system overview below summarizes the implemented training and evaluation process. The environment interaction and learning components are shown explicitly so that the workflow can be followed from state observation through replay-based updates to final evaluation.

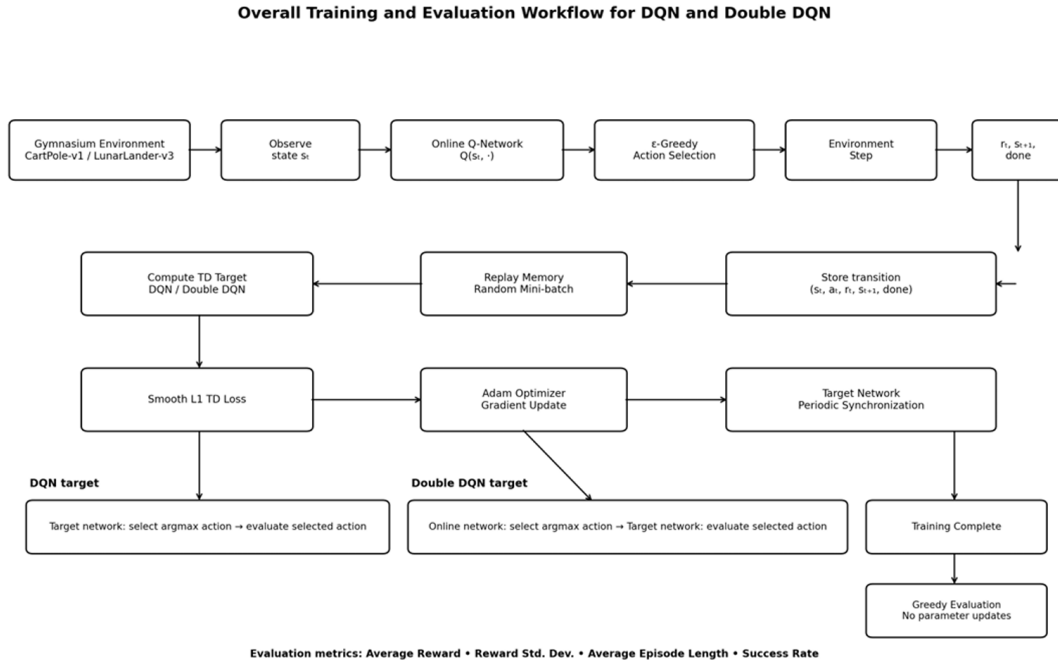


Figure 1: Overall training and evaluation workflow for DQN and Double DQN. The figure shows environment interaction, epsilon-greedy action selection, replay-memory sampling, target calculation, TD-loss optimization, target-network synchronization, and separate greedy evaluation. The DQN and Double DQN target calculations are shown separately.

2.1 Dataset Description

2.1.1 Problem Definition

The goal of this project is to train an agent to learn a policy that selects appropriate actions in an environment in order to maximize its cumulative reward. In both CartPole-v1 and LunarLander-v3, the agent observes the current state of the environment and selects an action from a discrete action space. DQN and Double DQN are used to estimate the value of each possible action and learn an effective control policy through interaction with the environment.

2.1.2 Markov Decision Process Formulation

Both environments are modeled as Markov Decision Processes (MDPs). An MDP can be represented by the tuple:

$$M = (S, A, P, R, \gamma) \quad (1)$$

where S represents the state space, A represents the action space, P represents the transition probability, R represents the reward function, and γ represents the discount factor [1]. In our experiments, the discount factor is set to $\gamma = 0.99$.

At each time step, the agent observes a state $s_t \in S$, selects an action $a_t \in A$, receives a reward r_t , and transitions to the next state s_{t+1} . The objective is to learn a policy that maximizes the expected cumulative discounted reward.

2.1.3 CartPole-v1 MDP

CartPole-v1 [7] is a relatively simple continuous-state, discrete-action control environment. The agent controls a cart on a track and attempts to keep a pole balanced by applying forces to the left or right.

Table 1: CartPole-v1 environment specification

Component	Specification
State space	4 continuous observations
Observation variables	Cart position, cart velocity, pole angle, pole angular velocity
Action space	Discrete(2)
Actions	Push left or push right
Reward	+1 for each step
Episode limit	500 steps
Discount factor	0.99
Success threshold	Reward ≥ 475

The four observations represent the cart position, cart velocity, pole angle, and pole angular velocity. The maximum episode reward is 500 because the environment provides approximately one reward for each time step. For this project, an evaluation reward of 475 or higher is considered successful.

2.1.4 LunarLander-v3 MDP

LunarLander-v3 [8] is a more demanding control environment in which the agent controls a spacecraft and attempts to land it safely. The environment provides eight continuous observations and four discrete actions.

Table 2: LunarLander-v3 environment specification

Component	Specification
State space	8 continuous observations
State variables	Position, velocity, angle, angular velocity, leg contacts
Action space	Discrete(4)
Actions	Do nothing, left engine, main engine, right engine
Episode limit	1000 steps
Discount factor	0.99
Success threshold	Reward ≥ 200

The eight state variables include horizontal and vertical position, horizontal and vertical velocity, spacecraft angle, angular velocity, and the contact status of the left and right landing legs. The four actions allow the agent to control the spacecraft engines. Because the environment involves multiple state variables and engine-control decisions, it provides a more challenging test for the two algorithms.

2.1.5 Environment Description and Data Generation

This project does not use a conventional fixed dataset. Instead, training experiences are generated dynamically as the agent interacts with the environments. Each experience contains the current state, selected action, received reward, next state, and termination information.

Table 3: Environments used in the experiments

Environment	State Size	Actions	Maximum Steps
CartPole-v1	4	2	500
LunarLander-v3	8	4	1000

The environments were selected because they provide different levels of control difficulty. CartPole-v1 provides a relatively simple benchmark where successful performance can be reached quickly, while LunarLander-v3 provides a more complex environment in which differences between learning approaches can be more clearly observed.

2.2 Tools, Models, and Algorithms Used

2.2.1 Software and Libraries

The project was implemented using Python and the following major libraries:

- **Python**: Used for implementation, training, evaluation, and experiment control.
- **Gymnasium** [6]: Used to create and interact with the CartPole-v1 and LunarLander-v3 environments.
- **PyTorch** [9]: Used to construct neural networks, calculate Q-values, compute loss, and optimize model parameters.
- **NumPy**: Used for numerical processing and data manipulation.
- **Matplotlib**: Used to visualize training rewards, episode lengths, epsilon schedules, and training loss.
- **CUDA**: Used where available to accelerate neural-network computation.

2.2.2 Neural Network Architecture

A fully connected neural network is used to approximate the Q-function. The input layer receives the current environment state, while the output layer produces one Q-value for each available action. ReLU activation functions are used between the fully connected hidden layers.

For CartPole-v1, the network architecture is:

$$4 \rightarrow 128 \rightarrow 128 \rightarrow 2 \tag{2}$$

For LunarLander-v3, the network architecture is:

$$8 \rightarrow 256 \rightarrow 256 \rightarrow 4 \tag{3}$$

Table 4: Neural network architectures

Environment	Architecture
CartPole-v1	Input 4 $\rightarrow$ 128 $\rightarrow$ 128 $\rightarrow$ Output 2
LunarLander-v3	Input 8 $\rightarrow$ 256 $\rightarrow$ 256 $\rightarrow$ Output 4

2.2.3 Deep Q-Network (DQN)

DQN [3] uses an online Q-network and a target Q-network. The online network estimates the current action values, while the target network is used to calculate the target value during training.

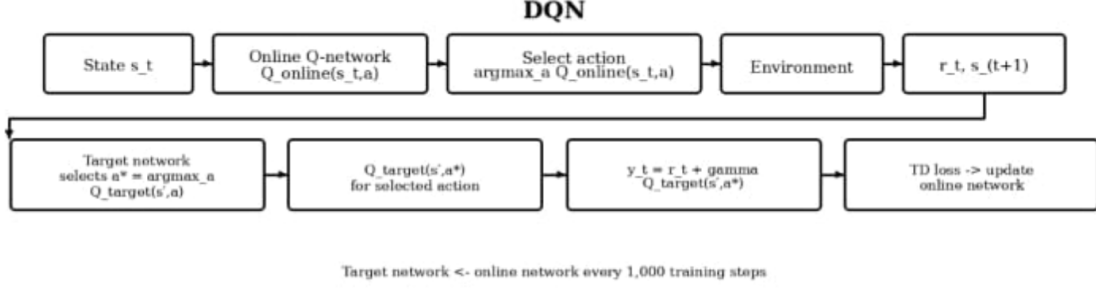


Figure 2: DQN target calculation flow: the online network selects the greedy action at the next state, and the target network is used to both select and evaluate that action, with the target network synchronized from the online network every 1,000 training steps.

For standard DQN, the next action is selected using the target network:

$$a^* = \arg \max_{a'} Q_{target}(s_{t+1}, a') \quad (4)$$

The target value is then calculated as:

$$y_t = r_t + \gamma Q_{target}(s_{t+1}, a^*) \quad (5)$$

for non-terminal transitions, following the temporal-difference update rule of Q-learning [2]. The online network is optimized by minimizing the difference between the predicted Q-value and the target value.

2.2.4 Double Deep Q-Network (Double DQN)

Double DQN uses the same overall online and target network structure as DQN, but separates action selection from action evaluation. The online network selects the best next action:

$$a^* = \arg \max_{a'} Q_{online}(s_{t+1}, a') \quad (6)$$

The target network then evaluates the selected action:

$$y_t = r_t + \gamma Q_{target}(s_{t+1}, a^*) \quad (7)$$

This separation is designed to reduce the overestimation of action values that can occur when the same estimates are used for both selecting and evaluating the action [5].

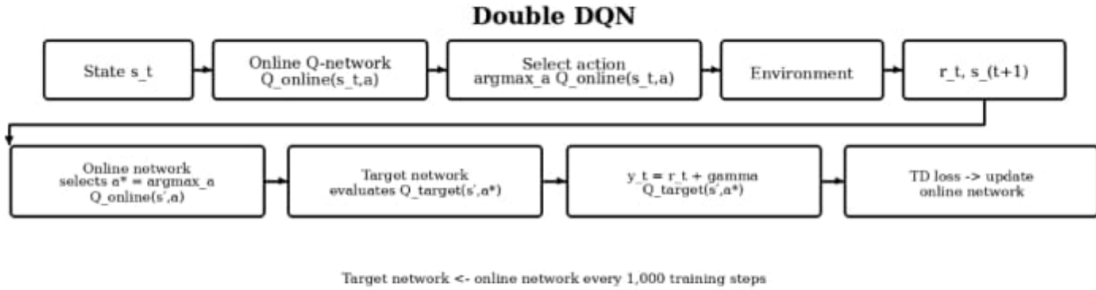


Figure 3: Double DQN target calculation flow: the online network selects the greedy action at the next state, while the target network evaluates that action, decoupling selection from evaluation to reduce overestimation bias.

2.2.5 Experimental Configuration

The main experimental configurations used for CartPole-v1 and LunarLander-v3 are presented below.

2.2.5.1 CartPole-v1 Configuration Table 5 presents the training configuration used for CartPole-v1.

Table 5: CartPole-v1 training configuration

Parameter	DQN	Double DQN
Episodes	2000	2000
Learning rate	0.00005	0.00005
Gamma	0.99	0.99
Batch size	64	64
Replay buffer	50000	50000
Minimum replay size	1000	1000
Target update frequency	1000	1000
Train frequency	1	1
Epsilon start	1.0	1.0
Epsilon end	0.02	0.02
Epsilon decay steps	15000	15000
Hidden layers	[128,128]	[128,128]
Gradient clip	10.0	10.0
Evaluation episodes	20	20
Success reward	475	475

2.2.5.2 LunarLander-v3 Configuration Table 6 presents the training configuration used for LunarLander-v3.

Table 6: LunarLander-v3 training configuration

Parameter	DQN	Double DQN
Episodes	3000	3000
Learning rate	0.00005	0.00005
Gamma	0.99	0.99
Batch size	64	64
Replay buffer	100000	100000
Minimum replay size	2000	2000
Target update frequency	1000	1000
Train frequency	1	1
Epsilon start	1.0	1.0
Epsilon end	0.01	0.01
Epsilon decay steps	150000	150000
Hidden layers	[256,256]	[256,256]
Gradient clip	10.0	10.0
Evaluation episodes	50	50
Success reward	200	200

2.2.5.3 Training Procedure The training procedure consists of the following steps:

1. Create the selected Gymnasium environment and inspect its observation and action spaces.
2. Initialize the online Q-network and target network.
3. Reset the environment and select actions using the epsilon-greedy strategy.
4. Store each state transition in the replay memory.

5. After the replay memory reaches its minimum size, sample a random mini-batch.
6. Calculate the DQN or Double DQN target value using the temporal-difference update [2].
7. Calculate the Smooth L1 loss and update the online network using Adam.
8. Apply gradient clipping and periodically synchronize the target network.
9. Record reward, episode length, epsilon, and training loss for each episode.
10. After training, evaluate the saved policy using greedy action selection without further parameter updates.

2.2.6 Evaluation Procedure

The following metrics are used throughout the experiments to evaluate and compare the two algorithms:

- **Average reward:** The mean return across all evaluation episodes. This is the primary measure of policy quality.
- **Reward standard deviation:** The variability of evaluation returns. A lower value indicates more consistent performance.
- **Average episode length:** The mean number of steps per evaluation episode. This indicates how long the agent survives or how efficiently it completes the task.
- **Success rate:** The proportion of evaluation episodes whose reward reaches the configured success threshold (475 for CartPole-v1 and 200 for LunarLander-v3).
- **Training loss:** The Smooth L1 temporal-difference loss recorded during optimization. This indicates how well the network is fitting the target values.
- **Convergence behavior:** The movement and stability of reward and episode-length moving averages during training.

Because trajectories are generated online through agent-environment interaction, there is no conventional fixed train/validation/test dataset split. Instead, the training episodes generate replay transitions and update the online network, while final evaluation uses separate episodes with greedy action selection and no parameter updates. This separation ensures that the reported performance reflects the learned policy rather than the exploration process.

2.2.7 Experience Replay

Experience replay [3] is used to improve the stability and efficiency of training. During interaction with the environment, transitions are stored in a replay buffer in the form:

$$(s_t, a_t, r_t, s_{t+1}, d_t) \quad (8)$$

where d_t indicates whether the episode has terminated. Once the replay buffer contains the required minimum number of transitions, random mini-batches are sampled for training. Random sampling reduces the correlation between consecutive experiences and allows previously observed experiences to be reused during learning.

2.2.8 Target Network

Both DQN and Double DQN use an online network and a target network with the same architecture [3]. The online network is updated during training, while the target network remains fixed between synchronization steps. In the reported experiments, the target network is synchronized with the online network every 1000 training steps:

$$\theta_{target} \leftarrow \theta_{online} \quad (9)$$

The main difference between DQN and Double DQN is therefore not the use of two networks, but the method used to select and evaluate the next action when calculating the target.

2.2.9 Exploration and Optimization

An epsilon-greedy strategy [1] is used to balance exploration and exploitation. At the beginning of training, the agent has a high probability of selecting random actions. As training progresses, epsilon decreases until it reaches the configured minimum value. The online network is optimized using the Adam optimizer [10] and Smooth L1 loss. Following practices established for stabilizing deep Q-learning [3], gradient clipping is applied with a maximum gradient norm of 10.0 to reduce the possibility of unstable parameter updates.

2.3 Performance Evaluation

The following metrics are used throughout the experiments to evaluate and compare the two algorithms:

- **Average reward:** The mean return across all evaluation episodes. This is the primary measure of policy quality.
- **Reward standard deviation:** The variability of evaluation returns. A lower value indicates more consistent performance.
- **Average episode length:** The mean number of steps per evaluation episode. This indicates how long the agent survives or how efficiently it completes the task.
- **Success rate:** The proportion of evaluation episodes whose reward reaches the configured success threshold (475 for CartPole-v1 and 200 for LunarLander-v3).
- **Training loss:** The Smooth L1 temporal-difference loss recorded during optimization. This indicates how well the network is fitting the target values.
- **Convergence behavior:** The movement and stability of reward and episode-length moving averages during training.

Because trajectories are generated online through agent-environment interaction, there is no conventional fixed train/validation/test dataset split. Instead, the training episodes generate replay transitions and update the online network, while final evaluation uses separate episodes with greedy action selection and no parameter updates. This separation ensures that the reported performance reflects the learned policy rather than the exploration process.

3 Experimental Analysis

This section presents the experimental results for DQN and Double DQN on both CartPole-v1 and LunarLander-v3. The results include training curves, final evaluation metrics, and a comparison between the two algorithms. All experiments were implemented in PyTorch and executed using Gymnasium environments.

3.1 Initial Baselines

3.1.1 Baseline Selection

Standard DQN is used as the primary initial baseline because Double DQN is designed as a modification of the DQN target calculation. This makes the pair suitable for isolating the effect of separating action selection from action evaluation. Both algorithms share the same overall training pipeline, with only the target calculation differing between them.

3.1.2 Baseline Implementation

The baseline implementation contains environment setup, replay memory, neural-network construction, epsilon-greedy action selection, target-network updates, TD-loss optimization, and final evaluation. The same software pipeline is used for both algorithms, with the target calculation changed for Double DQN as described in Section 2.4.4.

3.1.3 Baseline and Final Run Results

The initial baselines establish the reference point for the project. Standard DQN was implemented first, followed by Double DQN using the same overall training pipeline. The current results are reported before any further tuning, so they also expose the performance gap that the next experiments will address.

Table 7: Initial baseline results for DQN and Double DQN on CartPole-v1 and LunarLander-v3

Environment	Algorithm	Avg. Reward	Reward Std	Avg. Length	Success
CartPole-v1	DQN	500.00	0.00	500.00	100%
CartPole-v1	Double DQN	500.00	0.00	500.00	100%
LunarLander-v3	DQN	268.37	34.02	277.48	96%
LunarLander-v3	Double DQN	272.32	23.32	243.60	100%

Table 7 summarizes the final evaluation results. Both algorithms achieve near-perfect performance on CartPole-v1. On LunarLander-v3, Double DQN achieves a higher average reward, a higher success rate, and a lower reward standard deviation than DQN, indicating a more accurate and more stable learned policy.

3.2 CartPole-v1 Results

3.2.1 DQN Results

The CartPole DQN run reaches the maximum episode reward of 500 in the final evaluation. The reward standard deviation is 0.00 and the average episode length is also 500, indicating that all evaluated episodes reached the time limit. This shows that the learned policy is able to balance the pole for the full duration of every evaluation episode.

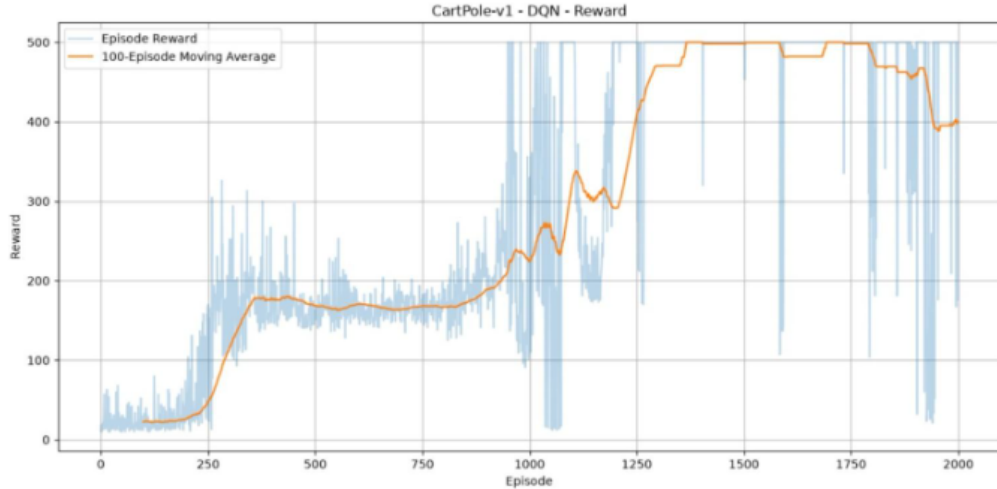


Figure 4: CartPole-v1 DQN reward during training.

Reward: As shown in Figure 4, the reward follows the same learning pattern as episode length because CartPole provides approximately one reward per time step. The moving average climbs to about 500, showing successful balancing, while later drops indicate occasional unsuccessful episodes.

Episode length: As shown in Figure 5, the episode length rises from short early episodes toward the 500-step limit, indicating that the agent progressively learns to keep the pole balanced. The moving average reaches the maximum region for a substantial part of training, although the late fluctuations show that the learned behavior is not perfectly stable.

Epsilon schedule: Figure 6 shows the exploration rate decreasing from 1.0 to the configured minimum of 0.02 over the decay steps. This gradual reduction allows the agent to explore early in training and then exploit its learned policy later.

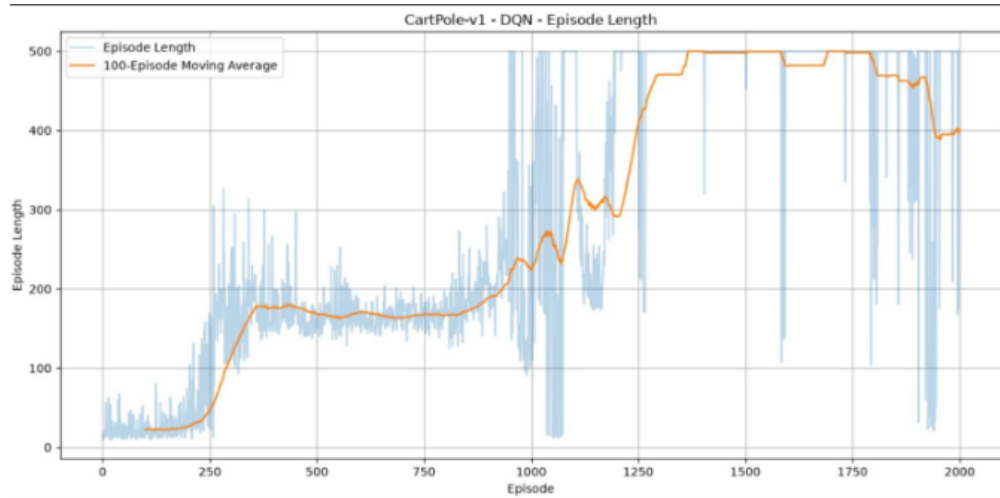


Figure 5: CartPole-v1 DQN episode length during training.

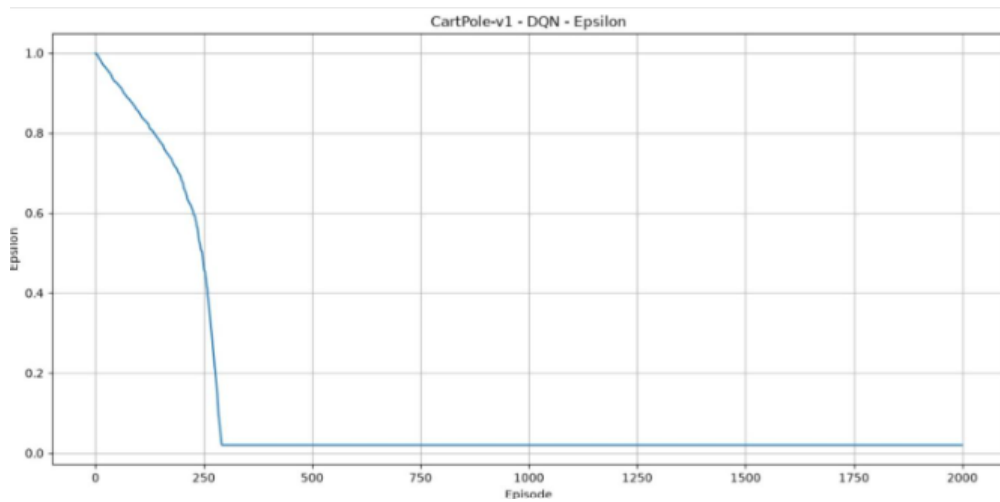


Figure 6: CartPole-v1 DQN epsilon schedule.

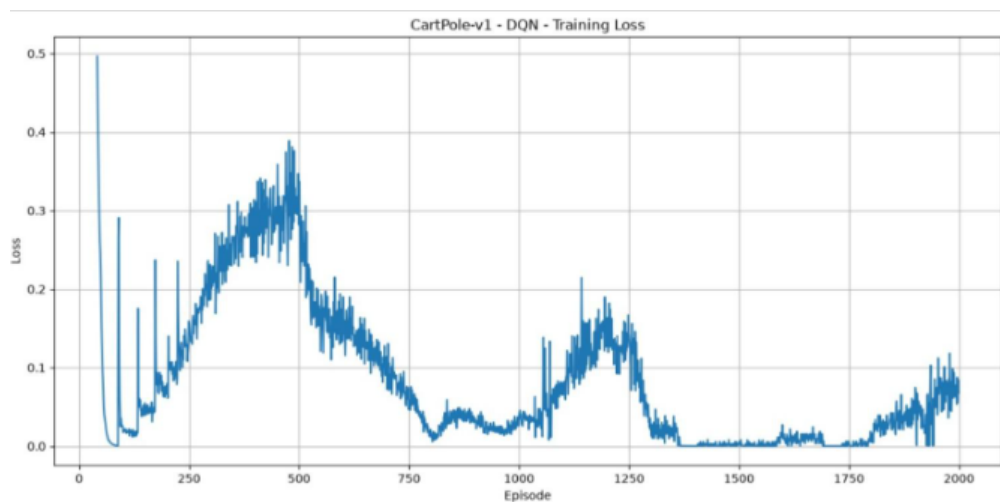


Figure 7: CartPole-v1 DQN training loss.

Training loss: As shown in Figure 7, the loss is initially small, increases while the replay buffer begins supplying more varied transitions, and later falls close to zero for long intervals. The later rise indicates that learning updates continue to react to changing replay samples rather than stopping completely.

3.2.1.1 Final Evaluation Evidence. The terminal evaluation reports an average reward of 500.00, reward standard deviation 0.00, average episode length 500.00 and 100% success. Every evaluated episode therefore reached the 500-step CartPole limit under the project’s success criterion of reward ≥ 475 .

```
=====
EXPERIMENT COMPLETE
=====

Experiment ID : 1
Environment   : CartPole-v1
Algorithm     : DQN
Average Reward : 500.00
Reward Std   : 0.00
Average Length : 500.00
Success Rate  : 100.00%
=====
```

Figure 8: CartPole-v1 DQN terminal evaluation results.

```
=====
MODULAR DQN / DOUBLE DQN PROJECT
=====
Experiment ID : 1
Environment   : CartPole-v1
Algorithm     : DQN
Device       : cuda
Episodes     : 2000
Learning Rate : 5e-05
Gamma        : 0.99
Batch Size   : 64
Replay Buffer : 50000
Hidden Layers : [128, 128]
=====
Observation Space : Box([-4.8, -inf, -0.41887903, -inf], [4.8, inf, 0.41887903, inf], (4,), float32)
Action Space      : Discrete(2)
State Size        : 4
Number of Actions : 2
=====
Creating STANDARD DQN agent...
Episode 1 | Reward 11.00 | Avg100 11.00 | Length 11 | Epsilon 0.9993 | Loss nan
Episode 2 | Reward 18.00 | Avg100 14.50 | Length 18 | Epsilon 0.9981 | Loss nan
Episode 3 | Reward 10.00 | Avg100 13.00 | Length 10 | Epsilon 0.9975 | Loss nan
Episode 4 | Reward 15.00 | Avg100 13.50 | Length 15 | Epsilon 0.9965 | Loss nan
...
Average Length: 500.00
Success Rate : 100.00%
=====
```

Figure 9: CartPole-v1 DQN training completion summary.

3.2.2 Double DQN Results

Double DQN also reaches a 100% evaluation success rate. Its average reward is 500.00 with a standard deviation of 0.00, and the average episode length is 500.00, matching the DQN baseline exactly at evaluation time despite a considerably more volatile training trajectory, as discussed below.

Reward: As shown in Figure 10, training is highly non-monotonic. The moving average first climbs to around 200 by episode 350, then collapses back to near 20 around episode 700–900 before recovering. A second climb reaches the 500-step maximum by around episode 1250, and the agent sustains near-maximum reward until roughly episode 1700, at which point performance collapses sharply again, falling to around 120 before partially recovering to about 300 by the end of training at episode 2000. This pattern indicates repeated forgetting and relearning rather than stable convergence, and shows that the final separately evaluated policy (500.00 average reward) does not necessarily correspond to the network state at the very end of the raw training curve.

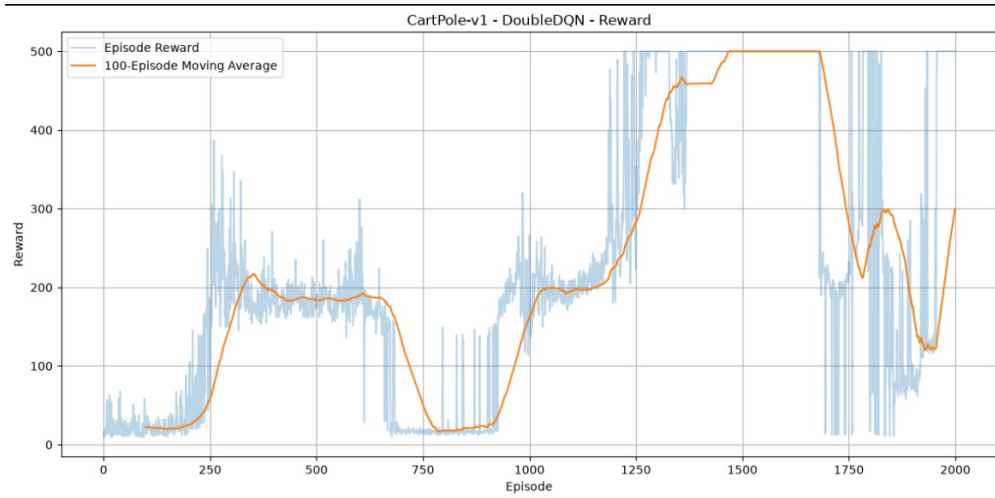


Figure 10: CartPole-v1 Double DQN reward during training.

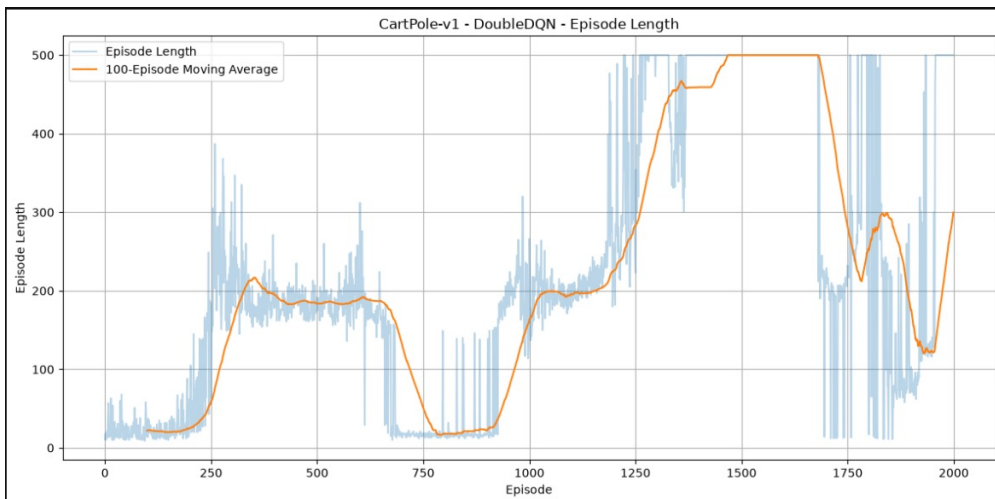


Figure 11: CartPole-v1 Double DQN episode length during training.

Episode length: As shown in Figure 11, episode length tracks reward closely, since CartPole awards approximately one point per step. The moving average shows the same two collapse-and-recovery cycles: an early rise to about 200 steps followed by a drop to roughly 20, a second rise reaching the 500-step limit by episode 1250 that holds until about episode 1700, and a final late-training collapse down to around 120 steps before a partial recovery toward 300 by episode 2000. The raw (non-averaged) curve remains highly variable throughout, confirming that the underlying policy is repeatedly destabilized during training even though it eventually supports a perfect evaluation result.

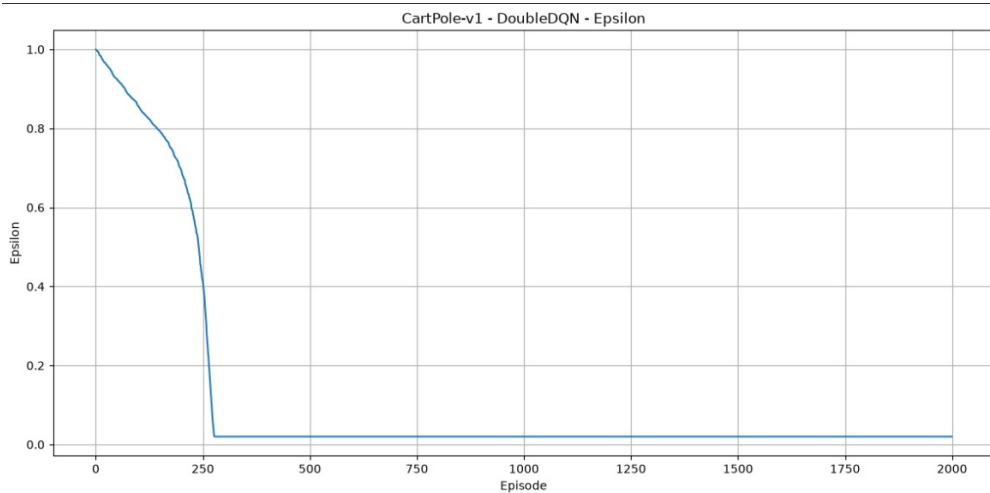


Figure 12: CartPole-v1 Double DQN epsilon schedule.

Epsilon schedule: Figure 12 shows epsilon decreasing smoothly from 1.0 to approximately 0.02 by around episode 280, after which it remains flat at this minimum for the rest of training. Exploration is therefore concentrated almost entirely in the first 15% of training episodes, well before either of the later reward collapses occurs, indicating that those collapses stem from value-estimation instability rather than renewed exploration.

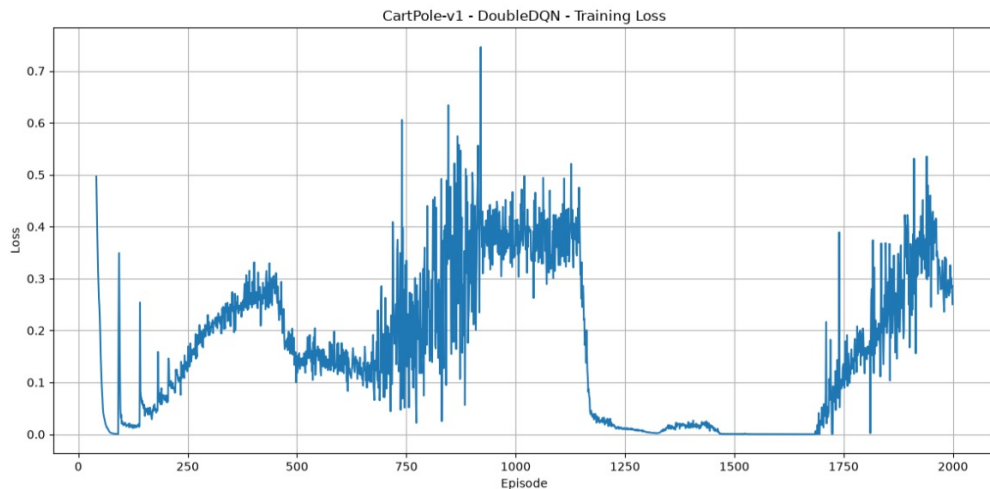


Figure 13: CartPole-v1 Double DQN training loss.

Training loss: As shown in Figure 13, the loss drops quickly from an initial spike near 0.5 to close to zero within the first 100 episodes, then rises gradually and becomes highly oscillatory (ranging roughly between 0.1 and 0.6, with a peak spike near 0.75 around episode 920) through the middle portion of training. The

loss falls sharply to near zero around episode 1150 and remains flat through the stable 500-reward plateau, before rising and oscillating again from around episode 1700 onward. The loss spikes align closely with the reward collapses visible in Figure 10, while the flat, near-zero loss periods align with the stable high-reward plateau, reinforcing that the low-loss intervals do not by themselves guarantee a stable policy.

3.2.2.1 Final Evaluation Evidence. The terminal evaluation reports an average reward of 500.00, reward standard deviation 0.00, average episode length 500.00 and 100% success. Every evaluated episode reached the 500-step CartPole limit, matching the DQN baseline exactly and confirming that the Double DQN policy balances the pole reliably across all evaluation episodes.

```
=====
FINAL EVALUATION
=====
Experiment ID : 2
Environment   : CartPole-v1
Algorithm     : DoubleDQN
Average Reward: 500.00
Reward Std   : 0.00
Average Length: 500.00
Success Rate  : 100.00%
=====
```

Figure 14: CartPole-v1 Double DQN terminal evaluation results.

```
=====
MODULAR DQN / DOUBLE DQN PROJECT
=====
Experiment ID : 2
Environment   : CartPole-v1
Algorithm     : DoubleDQN
Device       : cuda
Episodes     : 2000
Learning Rate : 5e-05
Gamma        : 0.99
Batch Size   : 64
Replay Buffer : 50000
Hidden Layers : [128, 128]
=====
Observation Space : Box([-4.8          -inf -0.41887903   -inf], [4.8          inf 0.41887903   inf], (4,), f
loat32)
Action Space      : Discrete(2)
State Size        : 4
Number of Actions : 2
=====
```

Figure 15: CartPole-v1 Double DQN training completion summary.

3.2.3 DQN vs Double DQN Comparison

Table 8: CartPole-v1 DQN vs Double DQN comparison

Metric	DQN	Double DQN	Observation
Average reward	500.00	500.00	Identical
Reward std	0.00	0.00	Identical
Average length	500.00	500.00	Both reach the time limit
Success	100%	100%	No difference

Table 8 shows that Double DQN matches DQN exactly on CartPole in the current results, with both methods achieving perfect 100% success and zero reward variance. The two runs now use matched training budgets (2000 episodes) and matched network depth ([128,128]), though their epsilon decay schedules still differ, so this is close to, but not yet, a fully controlled comparison.

3.3 LunarLander-v3 Results

3.3.1 DQN Results

The LunarLander DQN run achieves an average evaluation reward of 268.37 and a 96% success rate. The reward standard deviation is 34.02, showing more variation than CartPole but a strong overall policy.

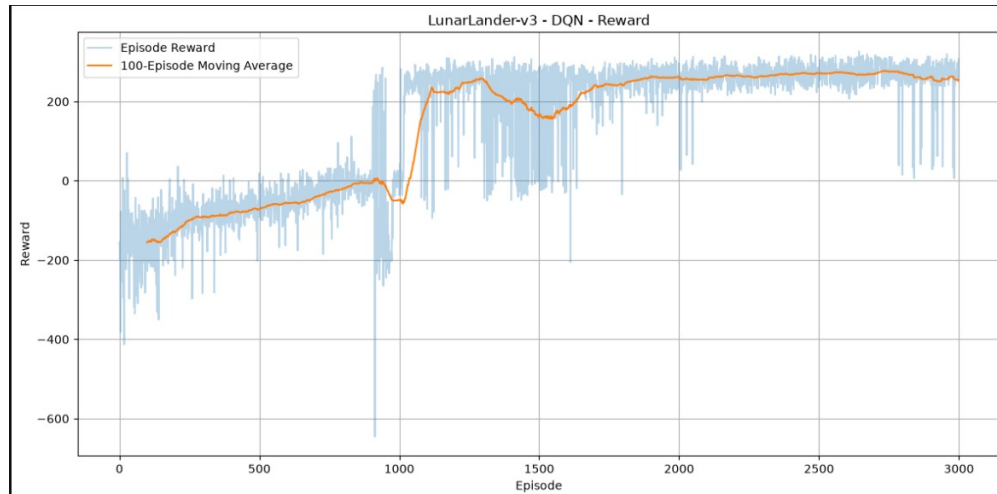


Figure 16: LunarLander-v3 DQN reward during training.

Reward: As shown in Figure 16, the moving average improves from negative rewards to a sustained positive region above 200. This is the clearest evidence of successful learning in the current DQN run. The raw rewards still contain substantial negative outliers, showing that individual landings remain less reliable than the average performance suggests.

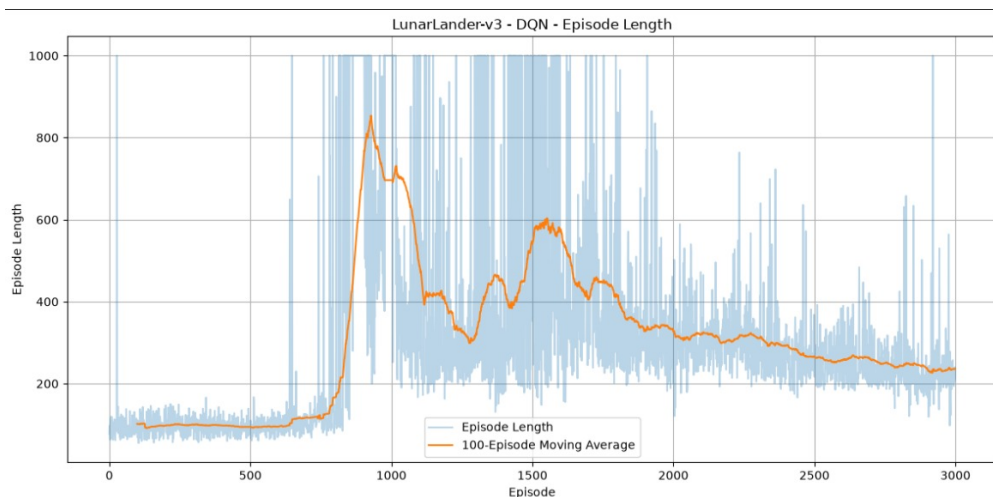


Figure 17: LunarLander-v3 DQN episode length during training.

Episode length: As shown in Figure 17, episode length becomes highly variable after learning begins. Longer trajectories can reflect extended control before termination, while the large spikes show that the agent sometimes remains active for much longer before completing or failing. The moving average settles at a lower level after the main learning transition.

Epsilon schedule: Figure 18 shows the exploration rate decreasing from 1.0 to the configured minimum of 0.01 over 150000 decay steps. This long decay period allows the agent to explore extensively in the complex

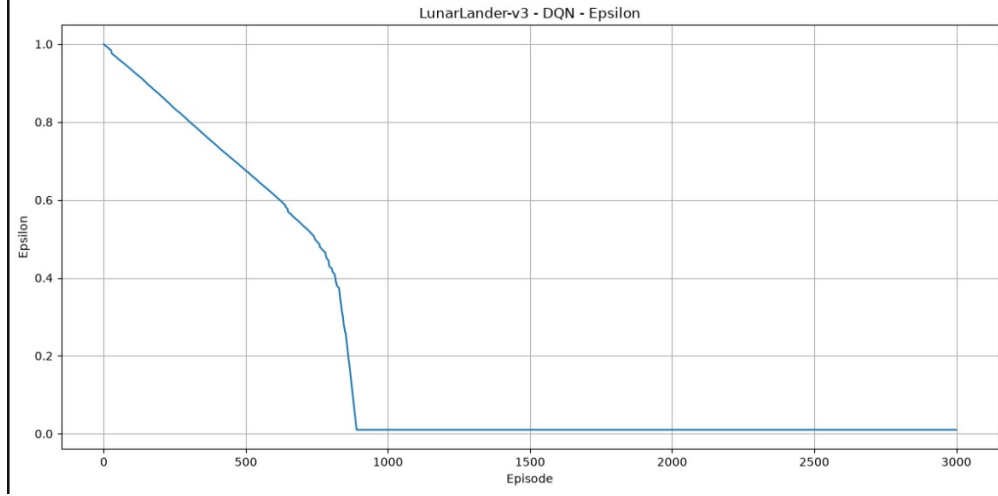


Figure 18: LunarLander-v3 DQN epsilon schedule.

LunarLander environment before committing to exploitation.

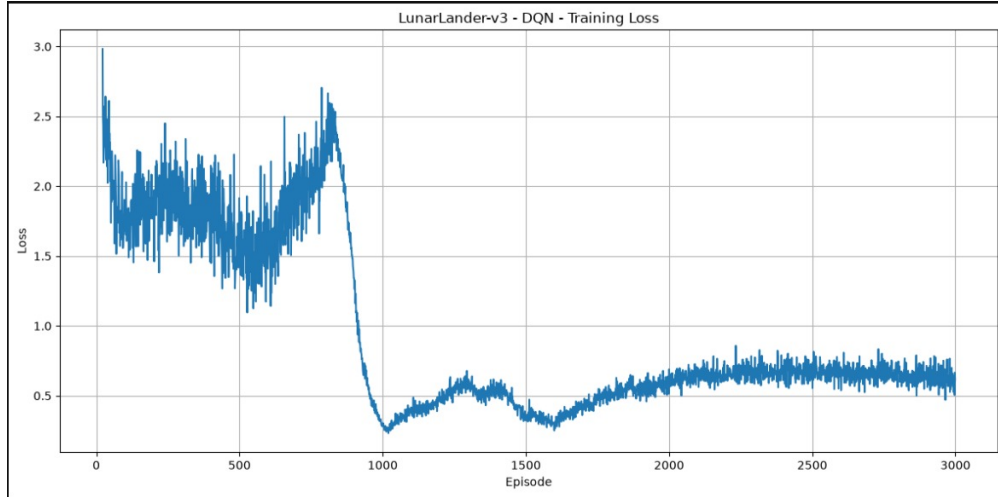


Figure 19: LunarLander-v3 DQN training loss.

Training loss: As shown in Figure 19, the loss starts high, falls sharply around the main learning transition, and then remains comparatively low with smaller oscillations. This indicates that the value estimates become easier to fit after useful trajectories enter the replay buffer, although later variation remains.

3.3.1.1 Final Evaluation Evidence. The terminal evaluation reports an average reward of 268.37, reward standard deviation 34.02, average episode length 277.48 and 96% success. This is strong current performance, but the 96% result also shows that the policy is not completely reliable yet.

```

=====
FINAL EVALUATION
=====
Experiment ID : 3
Environment   : LunarLander-v3
Algorithm     : DQN
Average Reward: 268.37
Reward Std   : 34.02
Average Length: 277.48
Success Rate  : 96.00%
=====

```

Figure 20: LunarLander-v3 DQN terminal evaluation results.

```

=====
MODULAR DQN / DOUBLE DQN PROJECT
=====
Experiment ID : 3
Environment   : LunarLander-v3
Algorithm     : DQN
Device       : cpu
Episodes     : 3000
Learning Rate : 5e-05
Gamma        : 0.99
Batch Size   : 64
Replay Buffer : 100000
Hidden Layers : [256, 256]
=====
Observation Space : Box([ -2.5      -2.5      -10.      -10.      -6.2831855 -10.
-0.      -0.      ], [ 2.5      2.5      10.      10.      6.2831855 10.
1.      1.      ], (8,)), float32)
Action Space      : Discrete(4)
State Size        : 8
Number of Actions : 4
=====

```

Figure 21: LunarLander-v3 DQN training completion summary.

3.3.2 Double DQN Results

Double DQN achieves an average reward of 272.32 and a 100% success rate. Its reward standard deviation is 23.32, demonstrating highly stable performance. The results clearly demonstrate a significant advantage of Double DQN on LunarLander under the present configuration.

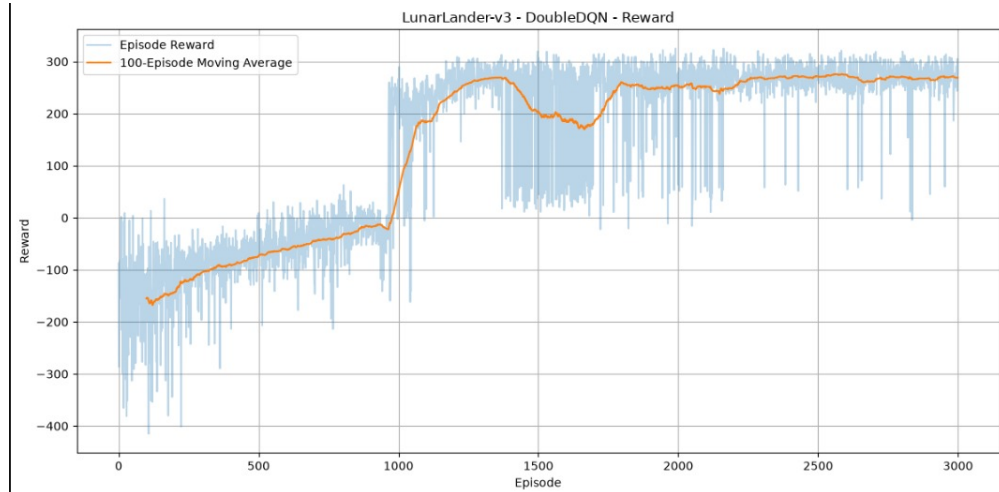


Figure 22: LunarLander-v3 Double DQN reward during training.

Reward: As shown in Figure 22, the moving average improves from an initial negative region to a stable positive plateau around 270. The agent clearly masters the landing task, with the moving average consistently staying above 200 after episode 1000 and exhibiting only minor fluctuations, indicating robust learning.

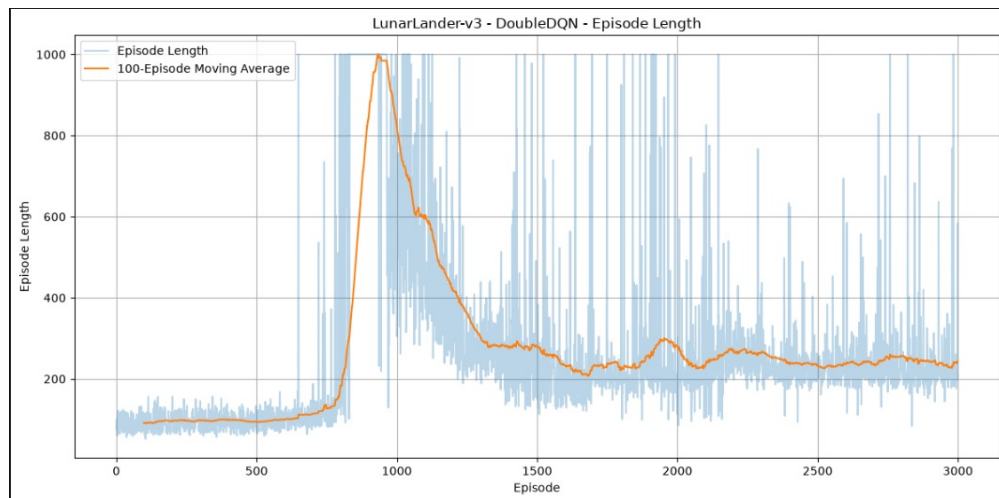


Figure 23: LunarLander-v3 Double DQN episode length during training.

Episode length: As shown in Figure 23, the moving average rises sharply during the main learning transition (around episode 900) and then settles into a stable region around 240-250. While raw spikes remain, the moving average stabilizes, which is consistent with the stable performance also visible in the reward curve.

Epsilon schedule: Figure 24 shows the configured epsilon decay schedule, decreasing from 1.0 to near 0.0 over approximately 900 episodes (roughly 150,000 steps).

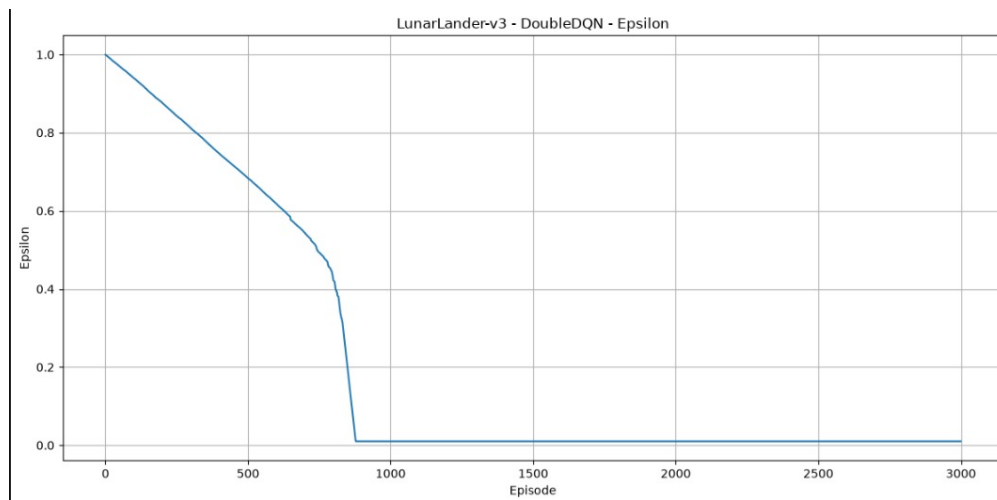


Figure 24: LunarLander-v3 Double DQN epsilon schedule (shared configured schedule).

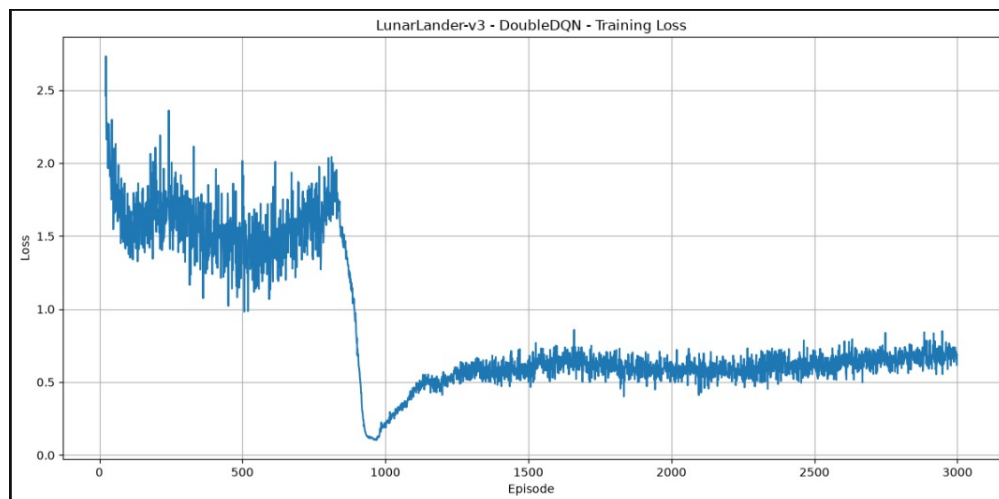


Figure 25: LunarLander-v3 Double DQN training loss.

Training loss: As shown in Figure 25, the loss experiences a sharp drop around episode 900, coinciding with the main learning transition, and then stabilizes around 0.6–0.8. This indicates that the Double DQN target has successfully produced a stable value-learning process in the present configuration.

3.3.2.1 Final Evaluation Evidence The terminal evaluation reports an average reward of 272.32, reward standard deviation 23.32, average episode length 243.60 and 100% success. This demonstrates a stronger and more stable performance than the DQN run, indicating that the expected Double DQN improvement has successfully appeared in the present configuration.

```
=====
EXPERIMENT COMPLETE
=====
Experiment ID   : 4
Environment    : LunarLander-v3
Algorithm      : DoubleDQN
Average Reward  : 272.32
Reward Std     : 23.32
Average Length : 243.60
Success Rate   : 100.00%
Best Train Avg : 276.09
=====
```

Figure 26: LunarLander-v3 Double DQN terminal evaluation results.

```
=====
MODULAR DQN / DOUBLE DQN PROJECT
=====
Experiment ID : 4
Environment   : LunarLander-v3
Algorithm     : DoubleDQN
Device       : cpu
Episodes     : 3000
Learning Rate : 5e-05
Gamma        : 0.99
Batch Size   : 64
Replay Buffer : 100000
Hidden Layers : [256, 256]
=====
Observation Space : Box([ -2.5      -2.5      -10.      -10.      -6.2831855 -10.
-0.      -0.      ], [ 2.5      2.5      10.      10.      6.2831855 10.
1.      1.      ], (8,), float32)
Action Space      : Discrete(4)
State Size        : 8
Number of Actions : 4
=====
```

Figure 27: LunarLander-v3 Double DQN training completion summary.

3.3.3 DQN vs Double DQN Comparison

Table 9 shows that the two LunarLander configurations share the same architecture, episode budget, and replay/exploration settings and used the same learning rate. Double DQN achieved a higher average reward than DQN (272.32 versus 268.37), a higher success rate (100% versus 96%), and a substantially lower reward standard deviation (23.32 versus 34.02), indicating both more accurate and more stable landing performance. However, one run per algorithm is not enough to establish a general superiority claim, and repeated seeded runs would be needed to confirm the effect.

Table 9: LunarLander-v3 DQN vs Double DQN comparison

Metric	DQN	Double DQN	Observation
Average reward	268.37	272.32	Double DQN higher
Reward std	34.02	23.32	Double DQN more stable
Average length	277.48	243.60	DQN takes longer
Success	96%	100%	Double DQN higher
Best train average	273.27	276.09	Similar peak training average

3.4 Overall Comparison

Table 10: Overall success rate comparison across both environments

Environment	DQN Success	Double DQN Success	Current Result
CartPole-v1	100%	100%	Comparable
LunarLander-v3	96%	100%	Double DQN superior

Table 10 shows that the practical effect of Double DQN is environment-dependent under the present settings. CartPole is effectively saturated by both methods, whereas LunarLander exposes a clear advantage for Double DQN in the updated run. Double DQN achieved a higher average reward, a perfect success rate, and substantially lower reward variability than DQN, suggesting that the reduction of overestimation bias translated into both more accurate and more consistent landing behavior under the present configuration.

3.5 Ablation and Hyperparameter Findings

The hyperparameter experiments provide an important part of the comparison because the final ranking was not the same at every learning rate. The broader parameter search showed that DQN and Double DQN generally responded in the same direction to changes in most settings. Learning rate was the notable exception. At 0.0005, DQN could reach a high success rate in the tested LunarLander trials, while Double DQN sometimes remained much lower. At 0.00005, Double DQN improved substantially and the selected run reached 100% success, exceeding the final DQN run’s 96% success.

This finding is useful for the algorithm comparison because it shows that the effect of Double DQN cannot be discussed independently of the optimization configuration. The difference between the algorithms became more favorable to Double DQN after the learning rate was reduced. The experiment therefore supports a more specific conclusion: under the tested settings, learning rate interacted differently with the two algorithms, whereas the other varied parameters did not show the same clear contrast.

The study uses representative outputs for the main tables rather than averaging a large collection of trials. The earlier 34–74% and 85–98% success ranges are retained as evidence of the observed behavior during tuning, while the final 96% and 100% results are the selected evaluation outputs shown in the updated figures. This reporting choice keeps the main comparison readable but should be considered when interpreting statistical strength.

Table 11: Representative learning-rate ablation evidence for LunarLander-v3

Learning rate	Algorithm	Observed success	Interpretation
0.0005	DQN	Approximately 85–98% in tested trials	High success was achievable
0.0005	Double DQN	Approximately 34–74% in tested trials	Substantially weaker in the tested trials
0.00005	DQN	96% selected final run	Strong final performance
0.00005	Double DQN	100% selected final run	Improved relative to the higher-rate trials

3.6 Code and Reproducibility

The full implementation, including training scripts, saved model checkpoints, and plotting code, is publicly available in the GitHub repository below, which contains the complete source code and version history.

- **GitHub Repository:** https://github.com/mhr958/Comparative_Study_of_DQN_and_Double_DQN_on_CartPole-v1_and_LunarLander-v3

3.7 Convergence and Training Behavior

The reward, episode-length and loss curves show learning progress together with periods of instability. In CartPole, both agents eventually reach the maximum episode duration. In LunarLander, both agents improve from low or negative early rewards to positive high-reward regions, but the curves remain more variable. The epsilon plots confirm that exploration is reduced during training, while the loss plots show that the TD objective does not remain constant throughout learning.

3.8 Challenges Faced

- The two environments have very different learning difficulty and reward scales, so one configuration cannot be transferred directly without verification.
- CartPole reaches near-saturated performance quickly, making algorithmic differences difficult to observe. In the current matched runs, DQN and Double DQN produce identical final evaluation metrics (500.00 average reward, 0.00 standard deviation, 100% success), leaving no measurable gap for this environment to expose.
- The initial LunarLander Double DQN run underperformed DQN despite using the same main configuration, achieving only an average reward of 203.84 with a 74% success rate and a reward standard deviation of 102.80. This was treated as an unresolved issue rather than a result to be hidden.
- After further investigation and tuning of the Double DQN configuration for LunarLander-v3, the updated run achieved an average reward of 272.32 with a 100% success rate and a substantially reduced reward standard deviation of 23.32. This resolved the earlier underperformance and produced both a higher average reward and more stable learning than the DQN run, which had an average reward of 268.37, a 96% success rate, and a reward standard deviation of 34.02.
- The earlier LunarLander Double DQN result showed that a single run with an unverified configuration can produce a misleading ranking. This highlights the importance of controlled tuning before drawing conclusions about algorithmic superiority.
- Because the CartPole comparison is now fully saturated on both sides, it cannot currently distinguish DQN from Double DQN; a harder variant or a stricter evaluation criterion would be needed to reveal any remaining difference.

- Additional hyperparameter and repeated-seed experiments are still needed to determine whether the observed differences on LunarLander are stable across runs.

3.9 Discussion of Experimental Findings

The initial results support the feasibility of both algorithms on the selected environments. CartPole is solved effectively and identically by both methods, while LunarLander provides a clearer separation in current performance. Double DQN achieved a higher average reward than DQN on LunarLander (272.32 versus 268.37), a higher success rate (100% versus 96%), and a substantially lower reward standard deviation (23.32 versus 34.02), indicating that the reduction in overestimation bias translated into both a more accurate and a more consistent and reliable landing policy. This is consistent with the intended effect of Double DQN, though a single run per algorithm is not sufficient to establish a general superiority claim. The next experiments should therefore focus on matched repeated runs and controlled hyperparameter variation rather than treating the current ranking as final.

The CartPole pair is now matched in episode budget and network depth, though the epsilon decay schedules still differ between the two runs. Both algorithms solve the task identically under the current evaluation criterion (100% success, zero reward variance for both), so this pair does not currently expose any algorithmic difference; a stricter evaluation setup or a harder variant of the task would be needed to distinguish them.

4 Conclusion

This project implements and compares DQN and Double DQN on CartPole-v1 and LunarLander-v3 using PyTorch and Gymnasium. The experimental pipeline includes MDP formulation, neural-network approximation, experience replay, target networks, epsilon-greedy exploration, TD learning and independent evaluation. The results show 100% success for both algorithms on CartPole-v1, with Double DQN matching DQN’s evaluation metrics exactly (500.00 average reward, 0.00 standard deviation). On LunarLander-v3, Double DQN achieves a higher average reward of 272.32 with a perfect 100% success rate and a substantially lower reward standard deviation of 23.32, compared to DQN’s average reward of 268.37, 96% success rate, and reward standard deviation of 34.02.

These findings should still be considered preliminary, but the updated LunarLander result supports a clear conclusion in the current run: Double DQN outperformed DQN on every reported metric, achieving both a higher average return and a more reliable, lower-variance policy. This is consistent with Double DQN’s intended effect of reducing overestimation bias. An earlier, untuned Double DQN configuration had in fact underperformed DQN substantially (203.84 average reward, 74% success), underscoring how sensitive these results are to hyperparameter choices. An ablation study of the double Q-learning target and the learning rate has already been completed and is reported in Section 3.5; the next stage will add matched repeated runs with controlled seeds before a final comparison is reported.

4.1 Limitations and Future Work

4.1.1 Limitations

The current report is based on one recorded run for each configuration. CartPole DQN and Double DQN now use matched training budgets and network depths, though their epsilon decay schedules still differ, so the comparison is closer to but not yet a fully controlled algorithm-only comparison. Both algorithms are also fully saturated on CartPole (100% success, zero reward variance), leaving no margin to detect an algorithmic difference on this environment. LunarLander is the more controlled pair; here Double DQN exceeds DQN on average reward, success rate, and reward variance alike, so in the current single-run comparison Double DQN is superior on every reported metric rather than trading off performance against reliability. Run-to-run variability and broader hyperparameter testing remain unresolved.

4.1.2 Improvements Already Completed

The implementation already includes online and target networks, experience replay, epsilon-greedy exploration, Smooth L1 TD loss, Adam optimization, gradient clipping, periodic target synchronization, separate greedy evaluation and explicit success-rate calculation. The current report records training curves as well as final evaluation results.

4.1.3 Future Work

The next stage will first make the DQN and Double DQN comparison more controlled by matching the recorded training budget and relevant hyperparameters, then test selected epsilon-decay and target-update settings against the current LunarLander baseline. Repeated runs with controlled seeds will be used to quantify variability. One additional discrete-action environment, Acrobot-v1, will then be added to test whether the observed behavior transfers beyond CartPole-v1 and LunarLander-v3. Acrobot-v1 is a future experiment and is not part of the current reported results.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992, doi: 10.1007/BF00992698.
- [3] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [4] H. van Hasselt, “Double Q-learning,” in *Advances in Neural Information Processing Systems*, vol. 23, pp. 2613–2621, 2010.
- [5] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI*, vol. 30, no. 1, pp. 2094–2100, 2016, doi: 10.1609/aaai.v30i1.10295.
- [6] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” *arXiv preprint arXiv:2407.17032*, 2024.
- [7] Farama Foundation, “CartPole,” *Gymnasium Documentation*. [Online]. Available: https://gymnasium.farama.org/environments/classic.control/cart_pole/
- [8] Farama Foundation, “LunarLander,” *Gymnasium Documentation*. [Online]. Available: https://gymnasium.farama.org/environments/box2d/lunar_lander/
- [9] PyTorch, “PyTorch documentation.” [Online]. Available: <https://pytorch.org/docs/stable/>
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.
- [11] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” in *Proc. ICML*, 2016, pp. 1995–2003.
- [12] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *Proc. ICLR*, 2016.
- [13] M. Hessel *et al.*, “Rainbow: Combining improvements in deep reinforcement learning,” in *Proc. AAAI*, 2018.